

숙제

`process-run.py` 프로그램은 프로세스가 실행되면서 변하는 프로세스의 상태를 추적할 수 있고, 프로세스가 CPU를 사용하는지(예, `add` 명령어 실행) 입출력을 하는지(예, 디스크에 요청을 보내고 완료되기를 기다린다)를 알아볼 수 있다. 상세한 사항은 README 파일을 확인하기 바란다.

문제

1. 다음과 같이 플래그를 지정하고 프로그램을 실행시키시오: `./process-run.py -l 5:100, 5:100`. CPU 이용률은 얼마가 되어야 하는가(예, CPU가 사용 중인 시간의 퍼센트?) 그러한 이용률을 예측한 이유는 무엇인가? `-c` 플래그를 지정하여 예측이 맞는지 확인하시오.
2. 이제 다음과 같이 플래그를 지정하고 실행시키시오: `./process-run.py -l 4:100, 1:0`. 이 플래그는 4개의 명령어를 실행하고 모두 CPU만 사용하는 하나의 프로세스와 오직 입출력을 요청하고 완료되기를 기다리는 하나의 프로세스를 명시한다. 두 프로세스가 모두 종료되는 데 얼마의 시간이 걸리는가? `-c` 플래그를 사용하여 예측한 것이 맞는지 확인하시오.
3. 옵션으로 지정된 프로세스의 순서를 바꾸시오: `./process-run.py -l 1:0, 4:100`. 이제 어떤 결과가 나오는가? 실행 순서를 교환하는 것은 중요한가? 이유는 무엇인가? (언제나처럼 `-c` 플래그를 사용하여 예측이 맞는지 확인하시오.)
4. 자, 다른 플래그에 대해서도 알아보자. 중요한 플래그 중 하나는 `-S`로서 프로세스가 입출력을 요청했을 때 시스템이 어떻게 반응하는지를 결정한다. 이 플래그가 `SWITCH_ON_END`로 지정되면 시스템은 요청 프로세스가 입출력을 하는 동안 다른 프로세스로 전환하지 않고 대신 요청 프로세스가 종료될 때까지 기다린다. 입출력만 수행하는 프로세스와 CPU 작업만 하는 프로세스 두 개를 실행시키면 어떤 결과가 발생하는가? (`-l 1:0,4:200 -c -S SWITCH_ON_END`)
5. 이번에는 프로세스가 입출력을 기다릴 때마다 다른 프로세스로 전환하도록 플래그를 지정하여 같은 프로세스를 실행시켜 보자 (`-l 1:0,4:100 -c -S SWITCH_ON_IO`). 이제 어떤 결과가 발생하는가? `-c`를 사용하여 예측이 맞는지 확인하시오.
6. 또 다른 중요한 행동은 입출력이 완료되었을 때 무엇을 하느냐이다. `-I IO_RUN_LATER`가 지정되면 입출력이 완료되었을 때 입출력을 요청한 프로세스가 바로 실행될 필요가 없다. 완료 시점에 실행 중이던 프로세스가 계속 실행된다. 다음과 같은 조합의 프로세스를 실행시키면 무슨 결과가 나오는가? (`./process-run.py -l 3:0, 5:100, 5:100, 5:100 -S SWITCH_ON_IO -I IO_RUN_LATER -c -p`) 시스템 자원은 효과적으로 활용되는가?
7. 같은 프로세스 조합을 실행시킬 때 `-I IO_RUN_IMMEDIATE`를 지정하고 실행시키시오. 이 플래그는 입출력이 완료되었을 때 요청 프로세스가 곧바로 실행되는 동작을

의미한다. 이 동작은 어떤 결과를 만들어 내는가? 방금 입출력을 완료한 프로세스를 다시 실행시키는 것이 좋은 생각일 수 있는 이유는 무엇인가?

8. 이제 다음과 같이 무작위로 생성된 프로세스를 실행시켜 보자. 예를 들면, `-s 1 -l 3:50,3:50`, `-s 2 -l 3:50,3:50`, `-s 3 -l 3:50,3:50`. 어떤 양상을 보일지 예측할 수 있는지 생각해 보시오. `-I IO_RUN_IMMEDIATE`를 지정했을 때와 `-I IO_RUN_LATER`를 지정했을 때 어떤 결과가 나오는가? `-S SWITCH_ON_IO` 대 `-S SWITCH_ON_END`의 경우에는 어떤 결과가 나오는가?